

CS210 Final Project

Pharmacy Database Management System

Using

Python and SQL

By Nikhil Aditya Vasantavada

GitHub

Link:<https://github.com/nav73-commits/Nikhil-Aditya-Vasantavada-CS210-Final-Project/tree/main>

Video Links:

YouTube Link

<https://youtu.be/KwwOMpJG58M>

Google Drive Link:

<https://drive.google.com/file/d/1f-RksQ0127q2m4iMJLZnUjiMkqG2wBTp/view?usp=sharing>

Project Definition

The Pharmacy Database Management System is designed to handle essential pharmacy operations such as data entry, retrieval, inventory tracking, sales management, customer records, and administrative control. It also includes functionality to determine the minimum required stock levels for medicines to avoid shortages. A string-based searching technique is implemented to allow users to retrieve information efficiently using identifiers such as Customer ID, Drug ID, or Pharmacy ID.

The system supports key pharmacy operations such as managing medicine stock, assisting in selecting appropriate medicines for customers, and maintaining accurate records of transactions and inventory. It also enables the generation of invoices, bills, and receipts while keeping track of supplier records. The overall aim of this project is to develop an efficient software solution for managing pharmaceutical store operations through a structured database system integrated with a Python-based application.

Problem Statement

Traditional pharmacy management systems often rely on manual record-keeping or disconnected software tools, which leads to inefficiencies such as inaccurate inventory tracking, prescription errors, and difficulty in maintaining unified patient and sales records. This project addresses these issues by developing a centralized database-driven pharmacy management system using Python and SQL.

The proposed system aims to improve accuracy in medicine dispensing, reduce stock shortages and overstock situations, and streamline daily pharmacy operations. It ensures that medicines, suppliers, customers, and prescriptions are managed in a structured and consistent manner, enabling efficient data retrieval and updates.

Entity-Relationship (ER) Diagram

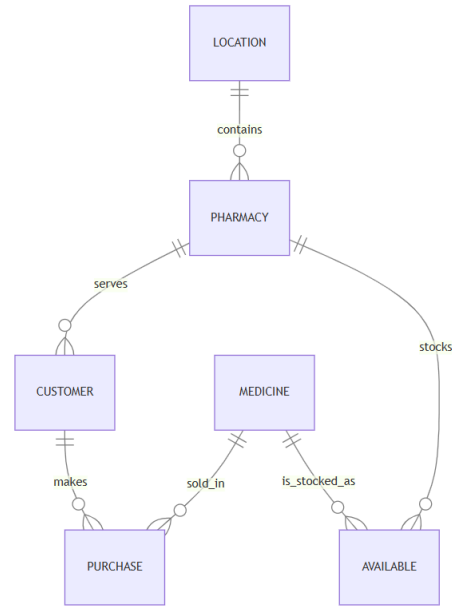


Figure 1: Relational Database Schema (3NF)

Connection to Course Material

This project directly applies core concepts from the Database Management Systems course. It demonstrates relational database design using entity-relationship modeling, schema creation, normalization techniques, and enforcement of data integrity constraints. These concepts ensure efficient storage, reduced redundancy, and improved consistency of data.

In addition, the project applies SQL operations such as insertion, deletion, updating, and querying of data. These operations reflect real-world database usage and reinforce theoretical concepts learned in class. The integration of Python with SQL further demonstrates how database systems are used in practical software applications.

Importance of the Project

This project is important because it improves the efficiency and accuracy of pharmacy operations, which are critical in healthcare environments. Errors in inventory management or prescription handling can directly impact patient safety and business performance.

By maintaining real-time records of inventory, prescriptions, and sales, the system supports better decision-making, such as identifying fast-moving medicines and predicting stock requirements. This helps pharmacies maintain optimal inventory levels and improve service quality.

Progress and Implementation

System Development and Data Utilization

The system development process follows a structured pipeline starting from data collection to implementation and testing.

Initial datasets for medicines, suppliers, and sample patient prescriptions were created and stored in a SQL database. The database schema was designed using normalization principles to ensure data consistency and reduce redundancy. Tables such as Medicines, Suppliers, Customers, Prescriptions, and Sales were created with appropriate primary and foreign key relationships.

A Python-based application was developed to interact with the database, allowing users to perform operations such as adding medicines, processing sales, and managing prescriptions. The system also includes features such as low-stock alerts, near-expiry detection, and sales summaries to support decision-making.

The system was evaluated through scenario-based testing, including stock updates, invalid

transactions, and expired medicine handling. These tests ensured that business rules were properly enforced and data integrity was maintained.

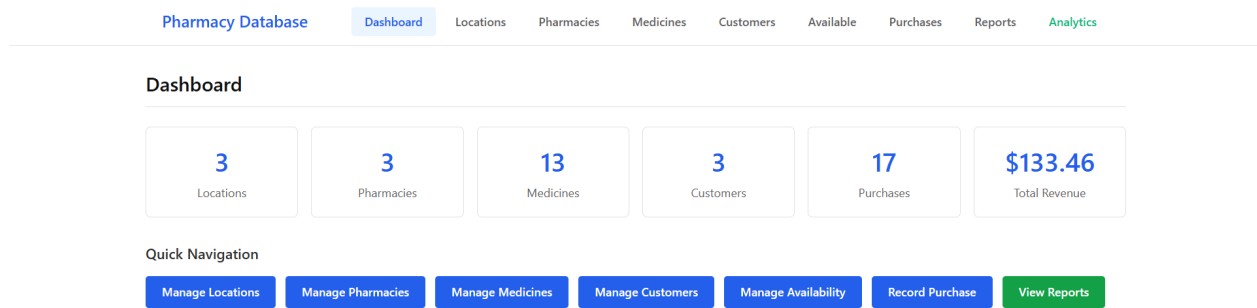


Figure 2: Administrative Dashboard

Data Collection, Cleaning, and Storage

The system uses multiple structured data sources:

- Transaction data (sales records, quantity, price, date)
- Inventory data (medicine details, batch number, expiry date)
- Prescription data (patient and doctor information, dosage details)
- Master data (categories, suppliers, and system users)

Data cleaning was performed to remove duplicates, handle missing values, and correct inconsistencies in formats such as dates and names. This ensured that the dataset was reliable and suitable for analysis.

For data storage, a relational SQL database was used. The database is normalized and designed to maintain referential integrity between tables. This structure enables efficient querying and

ensures consistency across all pharmacy operations.

Models, Techniques, and Algorithms

The system is built using the relational database model. An entity-relationship diagram was used to define relationships between entities before implementing the database schema.

Normalization techniques were applied to eliminate redundancy and improve data consistency. Standard SQL operations such as INSERT, UPDATE, DELETE, and SELECT were implemented to manage data effectively. These techniques ensure that the system remains efficient and reliable during real-world usage.

Experimental Design

The system was tested using real-world pharmacy scenarios such as adding medicines, updating stock levels, processing prescriptions, and retrieving records for verification. Each operation was evaluated to ensure correctness and consistency.

The testing process confirmed that the system can handle daily pharmacy tasks effectively while maintaining data accuracy. Any inconsistencies identified during testing were resolved through schema adjustments and validation improvements.

Key Findings and Results

The implementation significantly improved the efficiency of pharmacy data management. Operations such as searching for medicines, updating inventory, and generating reports became faster and more reliable.

The system also demonstrated strong data organization, ensuring that all records are stored in a structured format. This reduces the likelihood of errors and improves the overall reliability of pharmacy operations.

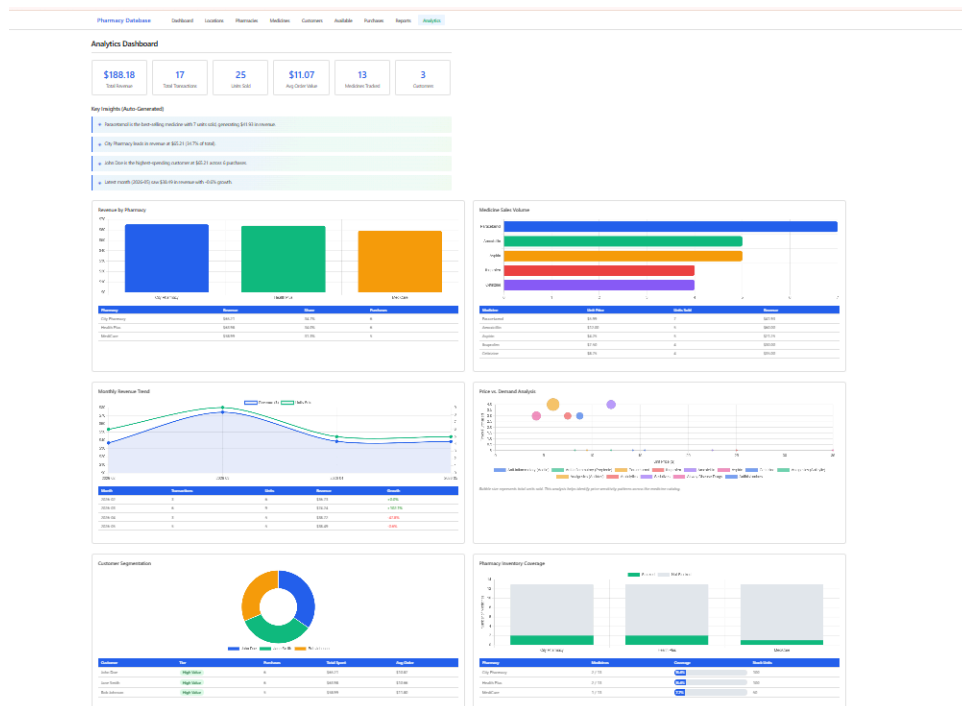


Figure 3: Pharmacy Performance, Medicine demand, and Customer spending patterns

Evaluation

The system performs efficiently for small to medium-sized datasets, making it suitable for typical

pharmacy environments. It provides fast data retrieval and accurate updates, ensuring smooth operation.

However, limitations include restricted scalability and limited support for multi-user environments. Advanced features such as analytics and external system integration are not included in the current version but can be considered for future improvements.

Methodology

The system was developed using a combination of relational database design and Python-based CRUD operations. Business rules were implemented to ensure stock validation, prevent invalid transactions, and maintain inventory accuracy.

A structured workflow was followed, including database design, application development, testing, and evaluation. Each stage contributed to building a reliable and functional system.

Machine learning techniques were considered for future enhancement, particularly for demand forecasting using historical sales data. Models such as regression or random forest could be used to predict future inventory requirements.

System accuracy was evaluated using validation rules and consistency checks to ensure correct stock calculations and prevent invalid operations.

Pharmacy Database

DashboardLocationsPharmaciesMedicinesCustomersAvailablePurchasesReportsAnalytics

Medicines

Add New Medicine

Medicine ID

e.g., MED001

Medicine Name

e.g., Paracetamol

Cost (\$)

e.g., 5.99

Dosage (mg)

e.g., 500

Add Medicine

All Medicines

Medicine ID	Medicine Name	Cost	Dosage (mg)	Actions
MED001	Paracetamol	\$5.99	500	EditDelete
MED002	Ibuprofen	\$7.50	200	EditDelete
MED003	Amoxicillin	\$12.00	250	EditDelete
MED004	Aspirin	\$4.25	100	EditDelete
MED005	Cetirizine	\$8.75	10	EditDelete
M01AB	Anti-inflammatory (Acetic)	\$15.50	50	EditDelete
M01AE	Anti-inflammatory (Propionic)	\$12.00	200	EditDelete
N02BA	Analgesics (Salicylic)	\$8.25	100	EditDelete
N02BE	Analgesics (Anilides)	\$9.50	500	EditDelete
N05B	Anxiolytics	\$25.00	10	EditDelete
N05C	Sedatives	\$22.50	5	EditDelete
R03	Airway Disease Drugs	\$35.00	100	EditDelete
R06	Antihistamines	\$14.25	10	EditDelete

Figure 4: Users can manage medicine records, update costs, and track dosage information through a user-friendly interface

Advantages and Limitations

The system is simple, efficient, and easy to use, making it suitable for basic pharmacy management tasks. It reduces manual errors and improves operational speed.

However, it lacks advanced features such as multi-user access, enhanced security mechanisms, and real-time analytics. These limitations can be addressed in future versions of the system.

Changes After Proposal

Differences from Initial Proposal

The initial proposal included more advanced features and complex system capabilities. However, due to time constraints and project scope, the focus was adjusted to prioritize core functionality and system stability.

The final implementation emphasizes essential pharmacy operations and ensures a clean, functional, and reliable system that demonstrates database concepts effectively.

Bottlenecks and Challenges

One of the main challenges was designing an efficient database schema that maintains proper relationships between entities while avoiding redundancy.

Another challenge involved ensuring that update and delete operations did not compromise data integrity. These issues were addressed through iterative testing and schema refinement, improving overall system reliability.

Conclusion and Future Work

Summary of Contributions

This project successfully implements a pharmacy management system that stores and manages detailed information about medicines, pharmacies, and customers. It enables users to search for medicines, track availability, and manage pharmacy operations efficiently.

The system also maintains detailed customer records including name, age, contact information, and purchase history, allowing better tracking of pharmacy transactions.

Workflow Efficiency

The system reduces the number of steps required to perform common pharmacy tasks such as adding medicines, processing prescriptions, and checking inventory. This improves operational efficiency and reduces redundancy in workflows.

By integrating database management with a user-friendly interface, the system demonstrates how structured data systems improve reliability and performance in real-world pharmacy environments.

Future Directions

Future improvements may include user authentication, barcode scanning, expiry tracking alerts, and enhanced reporting features.

The system can also be extended to support cloud-based storage and multi-user access, improving scalability and enabling real-time collaboration. With these enhancements, the system can evolve into a more complete and production-ready pharmacy management solution.